

N+1 문제 해결을 통한 성능 및 데이터 정합성 개선

데이터베이스 쿼리부터 가비지 컬렉션까지:
성능이 실제로 무너지는 지점

프록시 초기화 시점에서 발생한 추가 쿼리

테스트 코드

```
transactionList.forEach(transaction -> {  
    transaction.getWithdrawAccount();  
});
```

로그

```
Hibernate: insert into account_transaction (amount,created_at,deposit_account_id,deposit_account_balance,receiver,sender,tel,transaction_type,updated_at,  
withdraw_account_id,withdraw_account_balance,id) values (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, default)  
Hibernate: select t1_0.id,t1_0.amount,t1_0.created_at,t1_0.deposit_account_id,t1_0.deposit_account_balance,t1_0.receiver,t1_0.sender,t1_0.tel,t1_0  
.transaction_type,t1_0.updated_at,t1_0.withdraw_account_id,t1_0.withdraw_account_balance from account_transaction t1_0 where t1_0.withdraw_account_id=?  
offset ? rows fetch first ? rows only  
Hibernate: select a1_0.id,a1_0.balance,a1_0.created_at,a1_0.is_active,a1_0.number,a1_0.password,a1_0.updated_at,a1_0.user_id from account a1_0 where a1_0.id=?  
withdrawAccount 잔액: 1000  
withdrawAccount 번호: 1111111111
```

연관된 Account는 조회되지 않음 (Lazy Loading)

Account 조회를 위한 추가 쿼리 실행

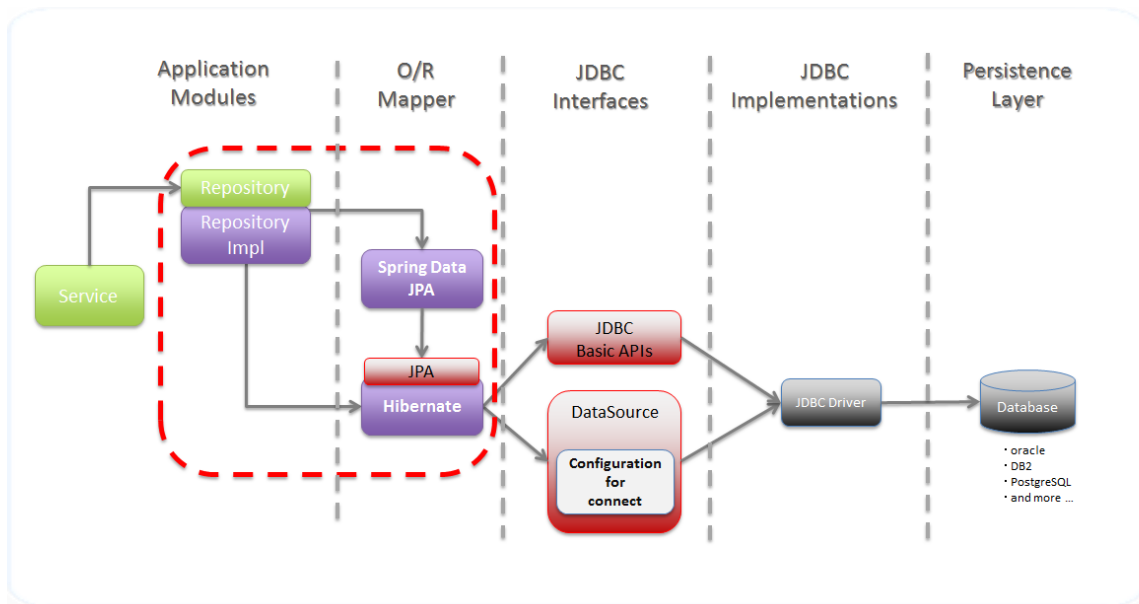
트랜잭션을 조회하는 시점에는 Account 엔티티가
함께 조회되지 않습니다.

이는 **JPA의 지연 로딩(Lazy Loading) 전략**으로
인해 연관 엔티티가 프록시 객체로 대체되었기
때문입니다.

getWithdrawAccount()에 접근하는 순간, 프록시 객체가
초기화되며 실제 Account 엔티티를 조회하는 추가
쿼리가 실행됩니다.

이 과정이 반복되면서 하나의 요청에서 다수의 쿼리가
발생하는 **N+1 문제**가 발생했습니다.

데이터 조회 흐름 분석과 N+1 문제의 구조적 원인



데이터 조회 흐름

개발자 코드 → JPA → Hibernate → JDBC → Database

JPA의 지연 로딩 전략이 Hibernate를 통해 실행되며 연관 엔티티 접근 시 추가 쿼리가 발생하는 구조입니다.

해당 구조로 인해 하나의 요청에서 다수의 JDBC 호출이 발생했고, 결과적으로 N+1 문제가 발생하는 원인이 되었습니다.

특히 ORM 추상화 계층 아래에서 실제 SQL 실행 흐름을 추적하며 문제를 분석한 점에서 단순 쿼리 최적화를 넘어 **구조적 원인**을 파악할 수 있었습니다.

Fetch join 적용하여 함께 로딩

테스트 코드

```
transactionList.forEach(transaction -> {  
    transaction.getWithdrawAccount();  
});
```

로그

```
Hibernate: insert into account_transaction (amount,created_at,deposit_account_id,deposit_account_balance,receiver,sender,tel,transaction_type,  
updated_at,withdraw_account_id,withdraw_account_balance,id) values (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, default)  
Hibernate: select t1_0.id,t1_0.amount,t1_0.created_at,t1_0.deposit_account_id,t1_0.deposit_account_balance,t1_0.receiver,t1_0.sender,t1_0.tel,t1_0  
.transaction_type,t1_0.updated_at,t1_0.withdraw_account_id,wa1_0.id,wa1_0.balance,wa1_0.created_at,wa1_0.is_active,wa1_0.number,wa1_0.password,wa1_0  
.updated_at,wa1_0.user_id,t1_0.withdraw_account_balance from account_transaction t1_0 join account wa1_0 on wa1_0.id=t1_0.withdraw_account_id where  
wa1_0.id=? offset ? rows fetch first ? rows only  
WithdrawAccount 잔액: 1000  
WithdrawAccount 번호: 1111111111
```

Fetch Join을 활용한 계좌 포함 거래 조회

JPQL에서 **join fetch**를 사용하여 Transaction 조회 시점에 연관된 Account 엔티티를 **함께 로딩**하도록 수정했습니다.

이로써 연관 엔티티가 프록시로 남지 않고 즉시 초기화되므로, 이후 필드에 접근하더라도 추가 쿼리가 발생하지 않게 됩니다.

N+1 vs Fetch Join

N+1 문제 발생

✓ Test Results	1 sec 178 ms
✓ TransactionRepositoryImplTests	1 sec 178 ms
✓ findTransactionList_withdraw_test	1 sec 178 ms



테스트 설정: 트랜잭션 1000배 수행

✓ Test Results	3 sec 46 ms
✓ TransactionRepositoryImplTests	3 sec 46 ms
✓ findTransactionList_withdraw_test	3 sec 46 ms

Fetch Join 적용(N+1 문제 해결)

✓ Test Results	1 sec 208 ms
✓ TransactionRepositoryImplTests	1 sec 208 ms
✓ findTransactionList_withdraw_test	1 sec 208 ms



✓ Test Results	2 sec 214 ms
✓ TransactionRepositoryImplTests	2 sec 214 ms
✓ findTransactionList_withdraw_test	2 sec 214 ms

	평균 지연 시간	메모리 영향	네트워크 I/O 횟수
N+1 Query	3,046 ms	Extreme Churn	N + 1
Fetch Join	2,214 ms	Stable/Low	1

- 평균 응답 시간: 3046ms → 2214ms 개선
- 쿼리 수 감소 (N+1 → 1)
- GC 부담 감소 (Object Churn 감소)
- 데이터 조회 시점 일관성 확보

불필요한 객체 생성을 줄여 성능을 안정적으로 개선했습니다.

N+1 문제가 성능을 망치는 이유

문제: 객체 생성이 급증하면서 GC가 이를 제때 처리하지 못해 성능 병목이 발생하는 구조

Object Churn

동일한 엔티티 객체가 반복 생성/삭제됨

Premature Promotion

Young 영역이 빨리 차면서 객체가 Old 영역으로
조기 이동

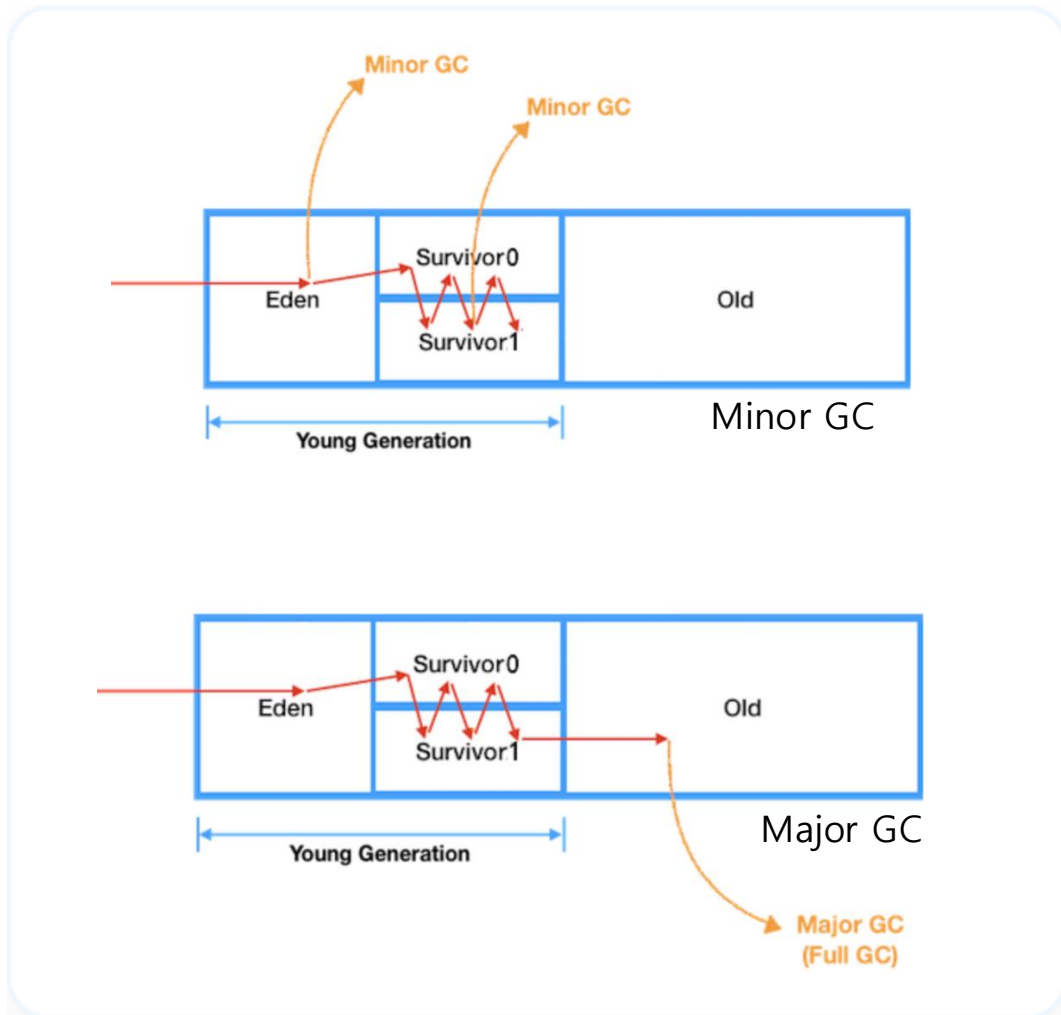
CPU 오버헤드 증가

GC 수행에 CPU 자원 소모

지연 시간 증가

네트워크 + GC 지연이 합쳐져 응답 악화

N+1이 JVM 메모리에 미치는 영향



N+1 문제로 인해 동일 엔티티 객체가 반복 생성되며 Heap 영역에서 Object Churn이 증가

- Young Generation에서 Minor GC 빈도 증가
- 일부 객체가 Old Generation으로 조기 승격
- Stop-The-World(STW) 발생 가능성 증가

결과적으로 GC 부하가 증가하며
서비스 응답 지연에 영향을 미칩니다.

결론: 메모리 관점에서의 최적화

현대에는 하드웨어 자원이 풍부하지만, GC 비용은 여전히 메모리 크기에 비례하여 증가합니다.

Heap 영역이 커질수록 Stop-The-World(STW)로 인한 비용 또한 증가하게 됩니다.

따라서 **N+1 문제 해결**은 단순한 SQL 최적화를 넘어,

JVM의 안정성을 위한 메모리 최적화 전략이라고 볼 수 있습니다.

감사합니다!

Author
신지윤

Connect
[linkedin.com/in/zzzyoonnn](https://www.linkedin.com/in/zzzyoonnn)